



Collider

Find the PRs that break
only when they meet.

PR #41

PASS

PR #42

PASS

+

#41 + #42

COLLISION

Before merge, not after.

First, what is a Pull Request?

A code change is reviewed before it joins the shared main branch.

01

Change code

A developer finishes a change.

02

Open a PR

They ask to merge it into shared code.

03

Review + checks

People and CI inspect the change.

04

Merge to main

Approved code joins the shared branch.

Collider checks the moment right before merge: when several PRs finally exist together.

PROBLEM

Each PR can be green. The problem appears between them.

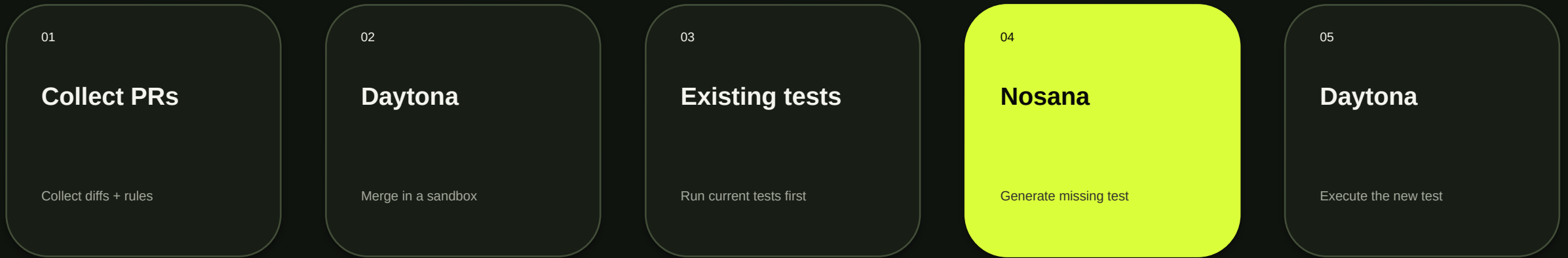
Coding agents create more PRs in parallel, while validation still happens mostly one at a time.



Both PRs can be correct on their own. If the interaction is missing from the test suite, the regression can still reach main.

Collider executes a future main that does not exist yet

Daytona executes the combined code. Nosana creates the missing interaction test.



The model does not decide the verdict. Runtime evidence does.

This is how Collider is used

Anyone can paste PR URLs and run a manual check.

01 PR INPUT

DAYTONA · NOSANA

Collider

DAYTONA · NOSANA · DNSIMPLE

Nosana 9.48 / 10

CI는 이 PR이 **오늘의 main**과 맞는지만 검사합니다. 곧 머지될 옆 PR과 함께 있어도 괜찮은지는 아무도 검사하지 않습니다.

열린 PR

currentjin/collider-demo#1

×

currentjin/collider-demo#2

×

currentjin/collider-demo#3

×

PR 추가

데모 3개 넣기

미래 충돌 검사

같은 저장소의 PR만 함께 검사합니다. 최소 두 개가 필요합니다.

02 FUTURE COLLISION MATRIX

검사 대상

#1
쿠폰 정액 할인 적용
src/priceCalculator.ts
test/priceCalculator.test.ts

#2
멤버십 등급 할인 적용
src/orderService.ts
test/orderService.test.ts

#3
청구서 부가세 10% 가산
src/invoice.ts
test/invoice.test.ts

미래 충돌 매트릭스

	#1	#2	#3
#1	—	COLLISION	검사 중
#2		—	● 대기
#3			—

#1 + #2 + #3

● 대기

● SAFE 함께 있어도 명세대로

● COLLISION 합쳤을 때만 어긋남

● 기존 단계에서 이미 걸림

● 대기 또는 건너뛸

COLLIDER · DAYTONA HACKSPRINT SEOUL

05

The existing tests passed.

The behavior still broke.

Collider runs the generated interaction test in Daytona and proves the difference.

01 RUNTIME VERDICT

#1 + #2

✓ git merge

충돌 없음

✓ 기존 테스트

5개 전부 통과

이 상호작용을 커버하는 테스트가 저장소에 없습니다. Collider 가 만듭니다

✗ 생성한 상호작용 테스트

실패

명세상 기대

실제

8450

9500

02 WHY IT COLLIDES

어디서 겹치는가

Thinking Process: 1. **Analyze the Request:** * Role: Senior Engineer analyzing code changes and writing tests (though output is restricted to JSON). * Task: An

Thinking Process: 1. **Analyze the Request:** * Role: Senior Engineer analyzing code changes and writing tests (though output is restricted to JSON). * Task: Analyze two independent Pull Requests (PR #1, PR #2) that were merged together without conflict but might violate specifications when combined. * Input: Specifications for fare settlement (quote calculation), discount policies (coupon fixed amount vs membership rate on base price), tax rules, current code changes in `priceCalculator.ts` and `orderService.ts`. * Constraint: Output **only** a JSON object with keys `collisionPoint`, `reason`

▼ 생성한 테스트 (저장소에 없던 코드)

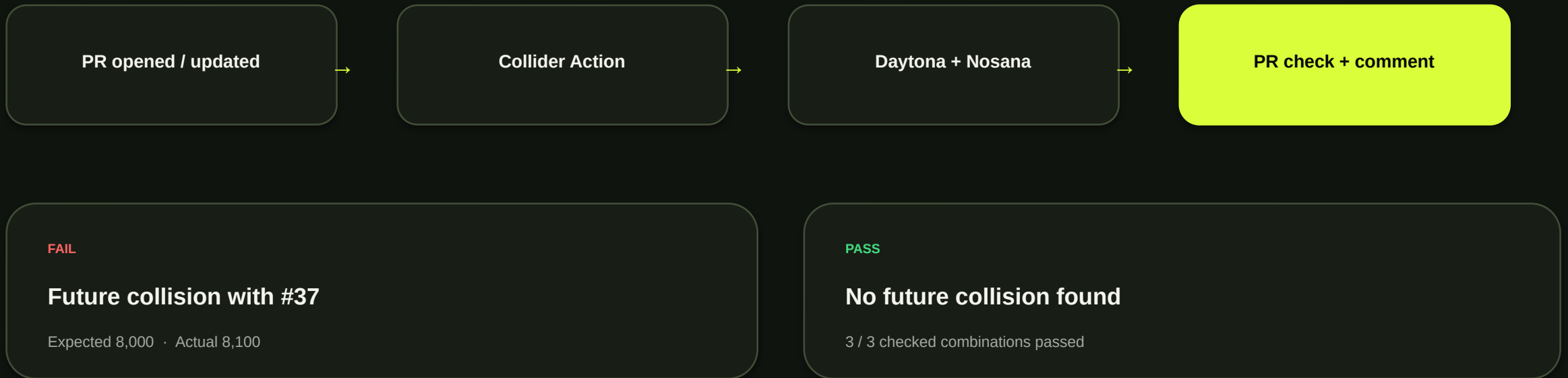
```
import { describe, expect, it } from 'vitest'
import { calculate } from '../src/priceCalculator.js'

describe('calculate', () => {
  it('쿠폰과 멤버십 할인이 모두 적용되더라도 기본 운임 기준으로 할인 계산 후 합산하여 차감한다.', () => {
    expect(
      calculate({ id: 'o3', basePrice: 10000, couponAmount: 500, membershipRate: 0.9 })
    ).toBe(8450)
  })
})
```

▶ 실행 출력

Try it once on the web. Run it automatically in CI.

The same engine runs whenever a pull request is opened or updated.



Reviewing one change is not the same as testing how changes interact.

Collider verifies the interactions that single-PR review and existing tests can miss.

CODE REVIEW

One PR

Review + existing checks

MERGE QUEUE

Several PRs

Existing tests

COLLIDER

Interactions between PRs

Existing + generated interaction tests

Catch PR collisions before they become production bugs.



Collider

각각은 정상인데,
함께 합치면 깨지는 PR 을 찾습니다.

PR #41

PASS

PR #42

PASS

+

#41 + #42

COLLISION

merge 하기 전에 , 실제로 합쳐봅니다.

먼저 , Pull Request 가 무엇일까요 ?

개발자가 만든 변경은 바로 서비스에 들어가지 않습니다 .

01

코드 수정

개발자가 기능을 수정합니다 .

02

PR 등록

' 이 변경을 합쳐도 될까요 ?' 라고 요청합니다 .

03

리뷰와 검사

사람과 CI 가 변경을 확인합니다 .

04

main 에 merge

승인된 변경이 공용 코드에 들어갑니다 .

Collider 는 마지막 단계 직전 , 여러 PR 이 실제로 함께 있을 때를 먼저 봅니다 .

PR 하나는 정상입니다 . 문제는 PR 사이에서 생깁니다 .

AI 코딩 에이전트가 동시에 더 많은 PR 을 만들지만 , 검증은 여전히 하나씩 이뤄집니다 .



각 PR 은 정상이어도 , 둘을 함께 쓰는 경우가 기존 테스트에 없으면 그 문제는 그대로 main 으로 들어갈 수 있습니다 .

Collider 는 아직 없는 미래의 main 을 실행합니다

Daytona 는 미래 코드 상태를 실행하고 , Nosana 는 빠진 상호작용 테스트를 만듭니다 .

01

PR 수집

diff 와 명세를 수집

02

Daytona

샌드박스에서 실제 merge

03

기존 테스트

현재 테스트 먼저 실행

04

Nosana

빠진 테스트 생성

05

Daytona

새 테스트를 실제 실행

AI 가 결론 내리지 않습니다 . 마지막 판정은 실제 런타임 결과입니다 .

실제로는 이렇게 사용합니다

누구나 PR URL 을 넣고 바로 검사할 수 있습니다 .

01 PR 입력

DAYTONA · NOSANA

Collider

DAYTONA · NOSANA · DNSIMPLE

Nosana 9.48 / 10

CI는 이 PR이 **오늘의 main**과 맞는지만 검사합니다. 곧 머지될 옆 PR과 함께 있어도 괜찮은지는 아무도 검사하지 않습니다.

열린 PR

currentjin/collider-demo#1

×

currentjin/collider-demo#2

×

currentjin/collider-demo#3

×

PR 추가

데모 3개 넣기

미래 충돌 검사

같은 저장소의 PR만 함께 검사합니다. 최소 두 개가 필요합니다.

02 미래 충돌 매트릭스

검사 대상

#1
쿠폰 정액 할인 적용
src/priceCalculator.ts
test/priceCalculator.test.ts

#2
멤버십 등급 할인 적용
src/orderService.ts
test/orderService.test.ts

#3
청구서 부가세 10% 가산
src/invoice.ts
test/invoice.test.ts

미래 충돌 매트릭스

	#1	#2	#3
#1	—	COLLISION	검사 중
#2		—	● 대기
#3			—

#1 + #2 + #3

● 대기

● SAFE 함께 있어도 명세대로

● COLLISION 합쳤을 때만 어긋남

● 기존 단계에서 이미 걸림

● 대기 또는 건너뛸

기존 테스트는 통과했습니다. 문제는 실제 동작에서 드러났습니다.

Collider 가 만든 상호작용 테스트를 Daytona 에서 실행해 차이를 증명합니다.

01 런타임 판정

#1 + #2

✓ git merge

충돌 없음

✓ 기존 테스트

5개 전부 통과

이 상호작용을 커버하는 테스트가 저장소에 없습니다. Collider 가 만듭니다

✗ 생성한 상호작용 테스트

실패

명세상 기대

실제

8450

9500

02 왜 충돌하는가

어디서 겹치는가

Thinking Process: 1. **Analyze the Request:** * Role: Senior Engineer analyzing code changes and writing tests (though output is restricted to JSON). * Task: An

Thinking Process: 1. **Analyze the Request:** * Role: Senior Engineer analyzing code changes and writing tests (though output is restricted to JSON). * Task: Analyze two independent Pull Requests (PR #1, PR #2) that were merged together without conflict but might violate specifications when combined. * Input: Specifications for fare settlement (quote calculation), discount policies (coupon fixed amount vs membership rate on base price), tax rules, current code changes in 'priceCalculator.ts' and 'orderService.ts'. * Constraint: Output **only** a JSON object with keys "collisionPoint", "reasoni

▼ 생성한 테스트 (저장소에 없던 코드)

```
import { describe, expect, it } from 'vitest'
import { calculate } from '../src/priceCalculator.js'

describe('calculate', () => {
  it('쿠폰과 멤버십 할인이 모두 적용되더라도 기본 운임 기준으로 할인 계산 후 합산하여 차감한다.', () => {
    expect(
      calculate({ id: 'o3', basePrice: 10000, couponAmount: 500, membershipRate: 0.9 })
    ).toBe(8450)
  })
})
```

▶ 실행 출력

웹에서 한 번 확인하고, 팀에서는 CI 로 자동화합니다

같은 분석 엔진이 PR 이 열리거나 바뀔 때 자동으로 실행됩니다 .



FAIL

Future collision with #37

Expected 8,000 · Actual 8,100

PASS

No future collision found

3 / 3 checked combinations passed

변경 하나를 보는 것과, 변경 사이를 보는 것은 다릅니다

Collider 는 코드 리뷰와 Merge Queue 가 놓치는 상호작용을 실제 실행으로 확인합니다 .

CODE REVIEW

PR 하나

리뷰 + 기존 검사

MERGE QUEUE

여러 PR

기존 테스트

COLLIDER

PR 사이의 상호작용

기존 테스트 + 새 상호작용 테스트

합치면 깨지는 PR 을 , 실제로 합치기 전에 찾습니다 .